



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

Uniform-Cost Search (UCS) selects the node with the lowest path cost $g(n)$, which is the actual cost from the starting node to the current node.

A* Search selects the node with the lowest total estimated cost $f(n)$, where $f(n) = g(n) + h(n)$. Here, $g(n)$ is the cost from the start to the current node, while $h(n)$ is the estimated cost from the current node to the goal.

Therefore, UCS considers only the cost already travelled, while A* considers both the cost already travelled and the estimated cost to reach the goal.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

Manhattan Distance is admissible for a 4-way movement grid because the agent can only move Up, Down, Left, or Right. The Manhattan Distance gives the minimum number of horizontal and vertical moves required to reach the goal when there are no obstacles. Therefore, it never overestimates the actual cheapest path cost.

The formula is:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

If the heuristic is not admissible and overestimates the actual cost to the goal, A^* may choose a more expensive path instead of the optimal path. Therefore, A^* may lose its guarantee of finding the shortest or optimal solution.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No, Manhattan Distance would not generally be an admissible heuristic for an 8-way movement grid if diagonal movement has a cost of 1.

This is because Manhattan Distance assumes that the agent must move horizontally and vertically. When diagonal movement is allowed, the agent can reach the goal using fewer moves than the Manhattan Distance predicts.

For example, if the agent is at (0, 0) and the goal is at (3, 3), Manhattan Distance is:

$$h(n) = |0 - 3| + |0 - 3| = 6$$

But with diagonal movement, the agent can reach (3, 3) in only 3 diagonal moves. Therefore, Manhattan Distance overestimates the actual cost and is not admissible.

For an 8-way grid where diagonal movement costs 1, a more suitable heuristic would be Chebyshev Distance:

$$h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

These answers follow the Practical 04 questions and its explanation of A^* , Manhattan Distance, and heuristic admissibility.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

A stronger heuristic for eating all remaining food can consider the distance between the agent and all remaining food items instead of considering only the closest food.

One suitable approach is to calculate the minimum distance needed to connect the agent with all remaining food items. For example, the heuristic can start with the Manhattan distance from the agent's current position to the nearest food, then add the distances between the remaining food items using a minimum spanning tree (MST).

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation



FACULTY OF COMPUTING